

Segment Tree

Fast Range Queries and Dynamic Updates

SHAYAN UN NOOR (2305024)
MD. SHADMAN SHAHRIYAR SHUVO (2305025)
MD. WASIF HAQUE (2305029)

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Dynamic Range Query Problem	3
1.3	Objective	3
2	The Range Query Problem	4
2.1	Problem Definition	4
2.2	Naïve Approach	4
2.3	Limitation of Prefix Precomputation	4
2.4	Need for a Dynamic Data Structure	5
3	Segment Tree	5
3.1	Basic Idea	5
3.2	Structure of a Segment Tree	5
3.3	Interval Representation	5
3.4	Stored Information and Merge Operation	5
4	Construction of a Segment Tree	6
4.1	Recursive Construction	6
4.2	Example	6
4.3	Construction Complexity	6
5	Point Update	7
5.1	Update Procedure	7
5.2	Example: update(4, 10)	7
5.3	Correctness of the Update	8
5.4	Complexity Analysis	8
6	Range Query	8
6.1	Three Cases	8
6.2	Query Procedure	8
6.3	Example: sum(3, 7)	9
6.4	Complexity Analysis	9
7	Generalization of Segment Trees	10
7.1	Range Sum, Minimum, Maximum, and GCD	10

7.2	Associative Merge Operations	10
8	Applications	10
8.1	Time-Series Data	10
8.2	Leaderboards	11
8.3	Scheduling and Booking	11
8.4	Other Dynamic Range Problems	11
9	Segment Tree vs. Other Data Structures	11
9.1	Naïve Array	11
9.2	Prefix Sum	11
9.3	Fenwick Tree	11
9.4	Comparison Table	12
10	Complexity and Practical Considerations	12
11	Conclusion	13

Segment Trees are hierarchical data structures designed to efficiently process range queries and dynamic updates over an array. A naïve approach may require linear time for every range query or update, which becomes inefficient when the number of operations is large. A Segment Tree addresses this problem by recursively partitioning an array into smaller intervals and storing a summary of each interval at a tree node. This organization allows both point updates and range queries to be performed in $O(\log n)$ time after $O(n)$ preprocessing. This report presents the structure and construction of a Segment Tree, explains its update and query procedures using worked examples, derives their time complexities, and discusses its generalization to operations such as minimum, maximum, and greatest common divisor. The report also compares Segment Trees with Fenwick Trees and other conventional approaches.

1 Introduction

1.1 Motivation

Many computational problems require repeatedly answering queries over a contiguous portion of a dynamically changing dataset. Consider a system tracking n financial instruments, where array A stores their current prices. At 9:31 AM, right after the market opens, 20,000 prices may change every second, while the system must still answer, instantly, “what is the total value of instruments l through r ?” The two fundamental operations required are therefore

$$\text{sum}(l, r) = \sum_{i=l}^r A[i] \tag{1}$$

and

$$\text{update}(i, v), \tag{2}$$

where $A[i]$ is modified to the value v .

1.2 Dynamic Range Query Problem

The difficulty arises from the interaction between queries and updates. If the array is scanned directly, a query in Equation (1) requires $O(n)$ time. If instead a running total is maintained, every update in Equation (2) requires $O(n)$ work to keep that total valid. Neither approach scales when both operations occur frequently and at large volume: keeping the sum fresh after every update costs $O(n)$ per update, while keeping updates instantaneous costs $O(n)$ per query. This tension — fast queries versus fast updates — appears in any large system that must *summarize a range and then change part of it*: stock feeds, leaderboards, and analytics dashboards all face it constantly.

1.3 Objective

This report presents the Segment Tree as a data structure that resolves this tension, achieving $O(\log n)$ time for both operations after an $O(n)$ preprocessing step. We derive its structure, construction, update and query algorithms, and their time complexities; work through a concrete numerical example; generalize the structure to other associative operations; and compare it against alternative approaches.

2 The Range Query Problem

2.1 Problem Definition

Let

$$A = [a_1, a_2, \dots, a_n] \quad (3)$$

be an array of n elements. A range-sum query is defined as

$$Q(l, r) = \sum_{i=l}^r a_i, \quad 1 \leq l \leq r \leq n, \quad (4)$$

and a point update changes a single element,

$$a_i \leftarrow v. \quad (5)$$

2.2 Naïve Approach

The most direct way to answer $Q(l, r)$ is to scan the range explicitly:

```
sum(l, r):  
    result = 0  
    for i = l to r:  
        result += A[i]  
    return result
```

For a range of length $r - l + 1$, this costs $O(r - l + 1)$ time, or $O(n)$ in the worst case:

$$T_{\text{query}}(n) = O(n). \quad (6)$$

A point update, by contrast, is trivially $O(1)$ under this scheme, since only $A[i]$ is written.

2.3 Limitation of Prefix Precomputation

An obvious optimization is to precompute prefix sums,

$$P[i] = \sum_{j=1}^i A[j], \quad (7)$$

so that any range sum can be answered in constant time via

$$Q(l, r) = P[r] - P[l - 1]. \quad (8)$$

This makes queries $O(1)$, but it introduces the opposite problem: whenever a single element $A[i]$ changes, every prefix sum $P[j]$ for $j \geq i$ becomes invalid, so an update costs $O(n)$ to restore correctness. This gives the fundamental trade-off summarized in Table 1.

Table 1: Trade-off between query and update cost across approaches.

Method	Query	Point Update
Naïve array	$O(n)$	$O(1)$
Prefix sum	$O(1)$	$O(n)$
Segment Tree	$O(\log n)$	$O(\log n)$

2.4 Need for a Dynamic Data Structure

Neither extreme is acceptable when queries and updates are interleaved at scale. What is needed is a structure that balances both costs rather than optimizing one at the expense of the other. The Segment Tree achieves exactly this balance by decomposing the array into a hierarchy of intervals and storing a summary for each one, so that both operations touch only $O(\log n)$ nodes instead of $O(n)$ or $O(1)$ -with-a-hidden- $O(n)$ -cost elsewhere.

3 Segment Tree

3.1 Basic Idea

A Segment Tree is a binary tree built over an array, where each node represents a contiguous sub-interval of the array and stores a summary value for that interval. The root represents the entire array; each internal node's interval is split into two halves, assigned to its left and right children; and each leaf represents a single array element. Instead of touching the whole array on every operation, the tree recursively splits the problem in half, solves each half, and combines the two answers — and only $O(\log n)$ splits are ever needed to reach any particular piece.

3.2 Structure of a Segment Tree

For an array of size n , the tree has height $h = \lceil \log_2 n \rceil$, and consequently $O(n)$ nodes in total (at most $\sim 2n$ for a complete binary tree representation). Each level of the tree partitions $[1, n]$ into disjoint, contiguous sub-intervals that together cover the whole array exactly once.

3.3 Interval Representation

Each node is identified by the interval $[lo, hi]$ it represents. If $lo = hi$, the node is a leaf corresponding to $A[lo]$. Otherwise, letting

$$mid = \left\lfloor \frac{lo + hi}{2} \right\rfloor, \quad (9)$$

the node's left child represents $[lo, mid]$ and its right child represents $[mid + 1, hi]$.

3.4 Stored Information and Merge Operation

For every node representing interval $[l, r]$, define the stored summary as

$$S(l, r) = \sum_{i=l}^r A[i]. \quad (10)$$

Given the midpoint $m = \lfloor (l + r)/2 \rfloor$, the summary satisfies the recursive relation

$$S(l, r) = S(l, m) + S(m + 1, r). \quad (11)$$

This is the *merge operation*: a parent's summary is obtained by combining its two children's summaries. For sum queries the merge is addition, but as shown in Section 7, any associative operation can be substituted in its place.

4 Construction of a Segment Tree

4.1 Recursive Construction

The tree is constructed recursively: the array interval is recursively divided until individual elements are reached, and internal-node summaries are then computed from their children during the return phase. The recursive calls themselves proceed top-down, but because a node's value is only written after both of its children's calls have returned, the *values* are effectively computed bottom-up.

```
build(node, lo, hi):
    if lo == hi:
        tree[node] = A[lo]
        return

    mid = (lo + hi) / 2

    build(left_child, lo, mid)
    build(right_child, mid + 1, hi)

    tree[node] = tree[left_child] + tree[right_child]
```

4.2 Example

Consider the running example used throughout this report:

$$A = [2, 1, 5, 3, 7, 4, 6, 8]. \quad (12)$$

The root represents $[1, 8]$, recursively divided into $[1, 4]$ and $[5, 8]$, then into $[1, 2]$, $[3, 4]$, $[5, 6]$, $[7, 8]$, with each leaf corresponding to one array element. Applying Equation (11) bottom-up gives

$$S(1, 2) = 2 + 1 = 3, \quad S(3, 4) = 5 + 3 = 8, \quad (13)$$

$$S(5, 6) = 7 + 4 = 11, \quad S(7, 8) = 6 + 8 = 14, \quad (14)$$

$$S(1, 4) = S(1, 2) + S(3, 4) = 3 + 8 = 11, \quad S(5, 8) = S(5, 6) + S(7, 8) = 11 + 14 = 25, \quad (15)$$

$$S(1, 8) = S(1, 4) + S(5, 8) = 11 + 25 = 36. \quad (16)$$

The resulting tree is shown in Figure 1.

4.3 Construction Complexity

Since every node is visited and computed exactly once, the number of operations satisfies the recurrence

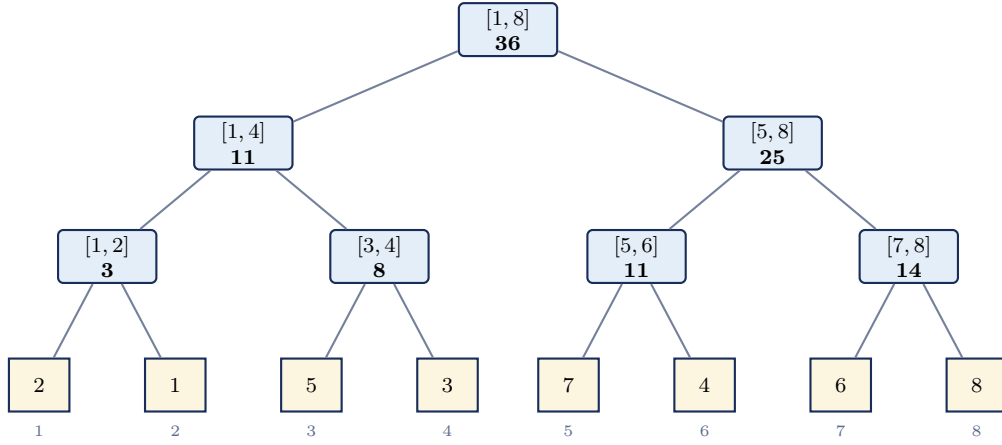


Figure 1: Segment Tree built over $A = [2, 1, 5, 3, 7, 4, 6, 8]$. Each parent stores the sum of its two children.

$$T(n) = 2T(n/2) + O(1), \quad (17)$$

which, by the Master Theorem, gives

$$T_{\text{build}}(n) = O(n). \quad (18)$$

The tree itself occupies $O(n)$ space.

5 Point Update

5.1 Update Procedure

An update replaces a single array value, $A[i] \leftarrow v$, and must restore correctness at every ancestor of leaf i . The procedure has two phases: a downward phase that locates the correct leaf, and an upward phase that recomputes each ancestor's summary.

Downward phase.

At a node representing $[lo, hi]$ with $mid = \lfloor (lo + hi)/2 \rfloor$, the target index i falls into exactly one half:

- if $i \leq mid$, recurse into the left child;
- otherwise, recurse into the right child.

This continues until a leaf $[i, i]$ is reached, whose value is set to v .

Upward phase.

As the recursion unwinds, every ancestor on the path recomputes its summary using Equation (11), combining its (possibly changed) two children.

5.2 Example: update(4, 10)

Starting from $A = [2, 1, 5, 3, 7, 4, 6, 8]$, consider `update(4, 10)`, which changes $A[4]$ from 3 to 10, giving

$$A = [2, 1, 5, 10, 7, 4, 6, 8]. \quad (19)$$

The downward path from the root follows

$$[1, 8] \rightarrow [1, 4] \rightarrow [3, 4] \rightarrow \text{index } 4, \quad (20)$$

since $4 \leq \text{mid} = 4$ at the root, $4 \leq \text{mid} = 2$ fails at $[1, 4]$ (so we go right into $[3, 4]$), and finally $4 \leq \text{mid} = 3$ fails at $[3, 4]$ (so we go right to the leaf).

On the way back up, each ancestor's summary is recomputed:

$$S(3, 4) = 5 + 10 = 15, \quad (21)$$

$$S(1, 4) = S(1, 2) + S(3, 4) = 3 + 15 = 18, \quad (22)$$

$$S(1, 8) = S(1, 4) + S(5, 8) = 18 + 25 = 43. \quad (23)$$

The root's stored value therefore changes from 36 to 43, matching the net change of +7 introduced by replacing 3 with 10.

5.3 Correctness of the Update

Correctness follows directly from Equation (11): only nodes whose interval contains index i can be affected by the change, and every such node lies on the unique root-to-leaf path to leaf i . Recomputing exactly this path, bottom-up, is therefore both necessary and sufficient to restore the invariant $S(l, r) = \sum_{i=l}^r A[i]$ at every node.

5.4 Complexity Analysis

Only a single root-to-leaf path is traversed, and the tree has height $h = \lceil \log_2 n \rceil$. Hence

$$\boxed{T_{\text{update}}(n) = O(\log n)}. \quad (24)$$

6 Range Query

6.1 Three Cases

To answer $Q(l, r)$, the query procedure compares the interval $[lo, hi]$ of the current node against the query range $[l, r]$ and falls into exactly one of three cases:

1. **Disjoint:** $[lo, hi] \cap [l, r] = \emptyset$. The node contributes nothing to the answer.
2. **Completely contained:** $[l, r] \supseteq [lo, hi]$. The node's stored summary can be returned immediately, with no further traversal.
3. **Partial overlap:** otherwise, both children must be explored recursively.

6.2 Query Procedure

Formally, for a node v representing $[lo, hi]$ with children v_L, v_R ,

$$Q(v, l, r) = \begin{cases} 0, & [lo, hi] \cap [l, r] = \emptyset, \\ S(v), & [lo, hi] \subseteq [l, r], \\ Q(v_L, l, r) + Q(v_R, l, r), & \text{otherwise.} \end{cases} \quad (25)$$

6.3 Example: $\text{sum}(3, 7)$

On the updated array $A = [2, 1, 5, 10, 7, 4, 6, 8]$, consider $Q(3, 7)$. Applying Equation (25) from the root, the query range decomposes into three fully-contained segments:

$$[3, 7] = [3, 4] \cup [5, 6] \cup [7, 7]. \quad (26)$$

Their stored/derived values are

$$S(3, 4) = 15, \quad S(5, 6) = 11, \quad A[7] = 6, \quad (27)$$

so that

$$Q(3, 7) = 15 + 11 + 6 = 32. \quad (28)$$

Table 2 traces every node visited during this query, following the classification rule in Section 5.1.

Table 2: Node-by-node decomposition of $Q(3, 7)$ on $A = [2, 1, 5, 10, 7, 4, 6, 8]$.

Visited segment	Status	Contribution
$[1, 8]$	Partial	—
$[1, 4]$	Partial	—
$[1, 2]$	Disjoint	0
$[3, 4]$	Contained	15
$[5, 8]$	Partial	—
$[5, 6]$	Contained	11
$[7, 8]$	Partial	—
$[7, 7]$	Contained	6
Total		32

As Table 2 shows, 8 of the tree's 15 nodes were visited to answer this query; the remaining 7 nodes (e.g. the leaves 1 and 2 beneath $[1, 2]$, and leaf 8 beneath $[7, 8]$) are pruned without further recursion once their ancestor is classified as disjoint or fully contained.

6.4 Complexity Analysis

At any level of the Segment Tree, the query interval has at most two boundary regions that can partially overlap tree segments — one near the left boundary and one near the right boundary. All other segments encountered at that level are either completely contained in the query interval or completely disjoint from it, and both of those cases terminate the recursion immediately by Equation (25). Consequently, at most $O(1)$ partially-overlapping nodes are expanded per level, and since the tree has $O(\log n)$ levels,

$$\boxed{T_{\text{query}}(n) = O(\log n)}. \quad (29)$$

7 Generalization of Segment Trees

One of the most powerful properties of the Segment Tree is that its *shape* never changes when the underlying problem changes — only what each node stores, and how two nodes are combined, needs to be redefined. The key requirement on the merge operation f is **associativity**:

$$f(f(a, b), c) = f(a, f(b, c)). \quad (30)$$

Addition satisfies this since $(a+b)+c = a+(b+c)$, and so do min and max, since $\min(\min(a, b), c) = \min(a, \min(b, c))$ and likewise for max. For a Segment Tree storing a single aggregate value, associativity of the merge is what makes the summary independent of how the interval was split; in implementations that must return a neutral value for a disjoint segment (Case 1 of Equation (25)), an appropriate *identity element* e satisfying $f(x, e) = x$ is also required — for example $e = 0$ for sum, or $e = +\infty$ for minimum. Table 3 lists several associative operations that can be substituted directly into the same tree structure.

Table 3: Segment Tree generalized to different associative merge operations.

Problem	Node stores	Merge rule
Range Sum	Sum	$a + b$
Range Minimum	Minimum	$\min(a, b)$
Range Maximum	Maximum	$\max(a, b)$
Range GCD	GCD	$\gcd(a, b)$
Any associative rule	Any summary	swap in the rule

7.1 Range Sum, Minimum, Maximum, and GCD

In each case, the build, update, and query algorithms of Sections 4–6 remain *structurally identical*; only the merge step (Equation (11)) is replaced. For range minimum, a node stores the smallest value in its interval and merges via min; for range maximum, the largest value via max; for range GCD, the greatest common divisor of the interval via gcd.

7.2 Associative Merge Operations

Because associativity is the only requirement, the same $O(n)$ build and $O(\log n)$ update/query bounds carry over unchanged to every operation in Table 3 — and to any other associative operation not listed. Mastering one structure therefore unlocks a broad class of range-query problems.

8 Applications

Table 4 summarizes representative domains where the Segment Tree’s core pattern — *summarize a range, then change part of it, repeatedly and at scale* — arises naturally.

8.1 Time-Series Data

In a system tracking financial instrument prices, the Segment Tree supports instant range-sum, range-min, or range-max queries over any contiguous time window, even as thousands of price updates arrive per second.

Table 4: Representative applications of the Segment Tree.

Application	Dynamic data		Possible query
Financial time series	Prices over time		Sum/min/max over a time interval
Leaderboards	Player scores		Range statistics over score intervals
Scheduling & booking	Time-slot availability		Aggregate availability over an interval
Analytics dashboards	Continuously changing measurements		Range minimum/maximum/sum

8.2 Leaderboards

The basic sum Segment Tree described in this report does not, by itself, answer rank or k -th-largest queries. However, the same tree structure can be *augmented* to store score frequencies at each node (rather than a running sum), which allows rank queries and k -th-order-statistic lookups to be answered in $O(\log n)$, with score updates handled just as fast.

8.3 Scheduling and Booking

Similarly, locating or reserving a free contiguous range of seats or time slots is not something the plain sum Segment Tree provides out of the box. It requires an *augmented* Segment Tree in which each node stores additional information — such as the count of free slots, and the longest prefix/suffix/interior run of free slots within its interval — so that availability queries and reservations can still be resolved in $O(\log n)$.

8.4 Other Dynamic Range Problems

More generally, any system that must repeatedly answer “what is true across this range?” while “something inside the range keeps changing” is a candidate for a Segment Tree, regardless of the specific domain.

9 Segment Tree vs. Other Data Structures

9.1 Naïve Array

A plain array supports $O(1)$ point updates but $O(n)$ range queries, since every query must be answered by scanning.

9.2 Prefix Sum

A prefix-sum array supports $O(1)$ range-sum queries but $O(n)$ point updates, since a single change invalidates every subsequent prefix.

9.3 Fenwick Tree

A Fenwick Tree (Binary Indexed Tree) supports both point update and range-sum query in $O(\log n)$, with a smaller memory footprint ($\sim n$) and a simpler implementation than a Segment Tree. A standard Fenwick Tree is particularly well suited to prefix/range aggregation under

operations such as addition, while Segment Trees provide a more general framework for associative range aggregates and accommodate range updates via lazy propagation considerably more naturally.

9.4 Comparison Table

Table 5 summarizes the comparison. Rather than declaring one structure to be strictly superior, the appropriate conclusion is that Fenwick Trees generally use less memory and have a simpler implementation, while Segment Trees support a broader class of associative range operations and naturally accommodate extensions such as lazy propagation.

Table 5: Segment Tree versus Fenwick Tree.

Property	Segment Tree	Fenwick Tree
Point update	$O(\log n)$	$O(\log n)$
Range query	$O(\log n)$	$O(\log n)$
Build	$O(n)$	$O(n)$
Typical memory	$\sim 4n$	$\sim n$
Range minimum/maximum	Flexible	More restricted
Lazy range update	Natural	Not the basic operation
Implementation	More complex	Simpler

10 Complexity and Practical Considerations

Table 6 consolidates the complexity results derived in Sections 4–6.

Table 6: Summary of Segment Tree time and space complexity.

Operation	Time Complexity	Explanation
Build	$O(n)$	Every tree node is constructed exactly once.
Point update	$O(\log n)$	Exactly one root-to-leaf path is recomputed.
Range query	$O(\log n)$	At most $O(1)$ partially-overlapping nodes are expanded per level (Section 6.4).
Space	$O(n)$	The tree has a linear number of nodes.

$$\begin{array}{l}
 \text{Build} = O(n) \\
 \text{Query} = O(\log n) \\
 \text{Update} = O(\log n) \\
 \text{Space} = O(n)
 \end{array} \tag{31}$$

As a purely illustrative, hypothetical operation-count comparison (not a benchmark of any real implementation): assuming a system with $n = 1024$ instruments processes $q = 36 \times 10^9$ combined queries and updates, and that each elementary step takes roughly 1 nanosecond, an $O(qn)$ naïve approach would require on the order of 10.24 hours of raw operation count, whereas an $O(q \log n)$ Segment Tree approach would require on the order of 6 minutes. The gap is driven

entirely by n versus $\log_2 n$ in the operation count, and the absolute times should be read only as an order-of-magnitude illustration rather than a measured runtime.

11 Conclusion

The Segment Tree resolves the fundamental tension between fast range queries and fast point updates by recursively decomposing an array into a hierarchy of intervals, each summarized at a tree node. This organization allows both operations to be performed in $O(\log n)$ time after an $O(n)$ build, as proven via the overlap argument for queries and the single-path argument for updates. Because the only requirement on the merge operation is associativity, the same structure generalizes immediately to range minimum, maximum, GCD, and a wide class of other problems, which is what makes the Segment Tree applicable across domains as different as financial time series, live leaderboards, and scheduling systems. Compared to a Fenwick Tree, a Segment Tree trades some memory and implementation simplicity for substantially greater flexibility, making the choice between the two ultimately dependent on what a given problem demands.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press.
- [2] A. Laaksonen, *Competitive Programmer's Handbook*.
- [3] cp-algorithms, "Segment Tree," *Algorithms for Competitive Programming*. https://cp-algorithms.com/data_structures/segment_tree.html